AIST4010 Lecture 15: Attention — Review Notes

1 Review of RNN++

Prior knowledge: text can be long, early observations may be important, some information is meaningless, and a logic break should reset the state.
GRU: the update gate interpolates between the candidate and the previous state,

$$H_t = (1 - Z_t) \odot H_{t-1} + Z_t \odot \tilde{H}_t,$$

with reset gates capturing short-term dependencies and update gates long-term ones.
LSTM: the cell state

$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$

adds a candidate instead of multiplying, and

$$H_t = O_t \odot \tanh(C_t)$$

reads the memory.
Deep RNN/LSTM/GRU: stacking layers increases capacity, with lower layers learning shorter-term dynamics and higher layers learning high-level trends. Bidirectional models see both directions but cannot do forward prediction and are very slow; they fit filling in missing words, token annotation, and sequence encoding.
Seq2seq: an encoder compresses the input into a context

$$c = q(h_1, \ldots, h_T),$$

and a decoder generates a variable-length output with cross-entropy per position.

2 Attention as neighbor pooling

Machine translation with a single hidden state cannot guarantee a proper representation for each word; the state flows across the sentence and must aggregate everything. Representation with neighbor information: an average

$$f(x) = \frac{1}{n} \sum_i y_i,$$

or a weighted average $f(x) = \sum_i \alpha(x, x_i)\, y_i$ with α related to similarity or distance. In the most general form, this is attention.

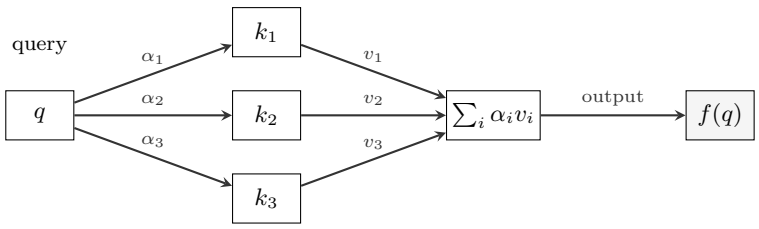


Figure 1: Attention pools the values through softmax weights: the query compares itself with every key, and the weighted values produce the output.

2.1 Attention for machine translation

The hidden state of each word considers interactions with all other words directly; a query hidden state gathers them through

$$f(x) = \sum_i \alpha(x, x_i)\, y_i.$$

The output hidden state is used for the downstream task, while the predefined state of each token carries its context and neighbor information. Three questions: how to encode characters (keys)? how to define the values? how to define the attention weight function?
Keys: one-hot encoding or a fully connected network over the one-hot vector. Values: a fully connected network over the one-hot vector.
Attention weights (dot-product):

$$\alpha(q, k_i) = \frac{\exp\big(a(q, k_i)\big)}{\sum_j \exp\big(a(q, k_j)\big)}, \qquad a(q, k) = \frac{q^\top k}{\sqrt{d}}.$$

Loss function: cross-entropy for each word prediction; training uses the common optimizers with back-propagation.

2.2 Self-attention

For each input x_i in the sentence,

$$y_i = \sum_j \alpha(x_i, x_j)\, v_j.$$

Each output is a context-aware hidden state for downstream classification or other tasks, attending against the sentence itself — including the token itself. All tokens can be computed in parallel; encoding and embedding are the most important things for NLP.

2.3 Position encoding

Attention pools without order information. Time series, sequences, and text all carry order, so positions must be injected explicitly. For $X \in \mathbb{R}^{n\times d}$, a matrix $P \in \mathbb{R}^{n\times d}$ adds sine and cosine features with frequencies that decay along the dimension:

$$p_{i,2j} = \sin\Big(\frac{i}{10000^{2j/d}}\Big), \qquad p_{i,2j+1} = \cos\Big(\frac{i}{10000^{2j/d}}\Big).$$

It looks like a weird function but people use it; it encodes absolute and relative positional information. The input dimension equals the raw encoding dimension plus the position encoding dimension.

2.4 Multi-head attention and the Transformer

- Input of an attention layer: $L\times$ (position embedding + one-hot); output: $L\times$ hidden-state dimension.
- To increase complexity: multi-head attention and stacking more attention layers.
- Multi-head attention projects queries, keys, and values into different representation subspaces, attends in each head, and concatenates the results.
- A Transformer block: multi-head attention learns the hidden state; fully connected layers control the dimension (for easy implementation); masked attention ignores padding positions; multiple blocks increase complexity.

3 Attention as a function approximator

- Fully-connected layers are a general function approximator: with enough nodes, layers, and non-linearity, any relation can be approximated (universal approximation theorem).
- A function is a mapping from a set X to a set Y assigning exactly one element of Y to each element of X. It can be represented by a mathematical formula or by a mapping table.
- Mathematical formula: memory efficient, but the form must be defined, and accuracy on the existing data depends on the form.

- Mapping table: precise for the existing data, but it must store the data, and predictions for unknown data are unclear — one can answer with neighbor information.
- Example: predict the gender of P5 (1.75 m, 70 kg) from a table of people; using neighbor information is the systematic way to make the prediction.

3.1 Nadaraya–Watson kernel regression

Attention is related to nonvolitional cues based on saliency and volitional cues that depend on the task; pooling neighbors with weights is the core operation,

$$f(x) = \sum_i \alpha(x, x_i)\, y_i.$$

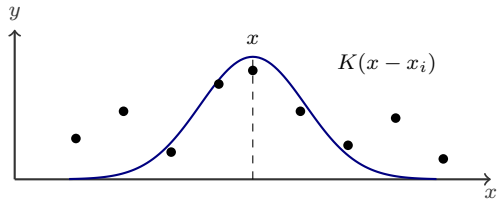


Figure 2: Nadaraya–Watson regression: a Gaussian kernel centered at the query x weights the nearby training points, and the normalized weights define the prediction.

Nadaraya–Watson regression normalizes a kernel over all neighbors:

$$f(x) = \frac{\sum_i K(x - x_i)\, y_i}{\sum_j K(x - x_j)}, \qquad K(u) = \frac{1}{\sqrt{2\pi}} \exp\Big(-\frac{u^2}{2}\Big).$$

With the Gaussian kernel $K(u) = (1/\sqrt{2\pi}) \exp(-u^2/2)$, the prediction becomes a softmax of squared distances:

$$f(x) = \frac{\sum_i \exp\big(-\frac{(x-x_i)^2}{2}\big)\, y_i}{\sum_j \exp\big(-\frac{(x-x_j)^2}{2}\big)}.$$

Attention with parameters: make the kernel width learnable,

$$f(x) = \frac{\sum_i \exp\big(-\frac{(x-x_i)^2}{2w^2}\big)\, y_i}{\sum_j \exp\big(-\frac{(x-x_j)^2}{2w^2}\big)}.$$

This is kernel smoothing whose band is fitted from data.

4 The attention pooling formula

The commonly used attention computes an output as a weighted combination of values, with weights from a softmax over a score function:

$$f(q, k_1, v_1, \ldots, k_m, v_m) = \sum_{i=1}^{m} \alpha(q, k_i)\, v_i, \qquad \alpha(q, k_i) = \mathrm{softmax}\big(a(q, k_i)\big) = \frac{\exp\big(a(q, k_i)\big)}{\sum_j \exp\big(a(q, k_j)\big)}.$$

Different score functions a lead to different attention behaviors.

4.1 Additive attention

The score combines query and key through a small network,

$$a(q, k) = w_v^\top \tanh\big(W_q q + W_k k\big).$$

It is usually used when queries and keys are vectors of different lengths.

4.2 Scaled dot-product attention

The score is the dot product scaled by $\sqrt{d}$,

$$a(q, k) = \frac{q^\top k}{\sqrt{d}}.$$

It is usually used when queries and keys are vectors of the same length. How to improve it: multi-head attention projects queries, keys, and values into different representation subspaces and pools in each head.

4.3 Self-attention with position encoding

Applied to a sequence, the embedding of each token is

$$y_i = f(x_i, x_1, x_1, \ldots, x_n, x_n).$$

There is no order information in the pooling, so position encoding is added with the sine/cosine scheme of Section 2.3.

5 Transformer

Putting everything together: token embeddings plus position encoding, stacked blocks of multi-head self-attention and fully connected layers, with masked attention that ignores padding positions. After learning the hidden states (embeddings), the Transformer supports machine translation, token-level prediction, sentence-level prediction, text generation, and more. With labelled data it is supervised learning with cross-entropy loss. If we only have a corpus without labels, what should the loss be? This motivates self-supervised objectives such as masked language modeling and next-token prediction, the topic of the next lecture.